

TradingAgents 小白指南

一个让 LLM 组建"虚拟交易公司"互相辩论再做决策的多智能体框架

★ 109,521 stars

Python • v0.5.2

LangGraph 编排

12 个智能体角色

arXiv 2412.20138

7 家免费数据源

⚠️ 免责声明 (先读这个)

TradingAgents 官方明确定位为研究框架，不构成任何投资建议。它的输出 (BUY/SELL/Overweight) 只是 LLM 基于公开数据的"文本判断"，没有仓位、现金账本、撮合成本模型，历史上也不保证任何收益。本指南的所有实验都是离线假数据 + 脚本假 LLM 的纯技术验证，不接真实行情、不接真实券商、不花一分钱。股市有风险，实盘需谨慎。

Q1 一句话是什么

Q2 为何模仿交易公司

Q3 12角色怎么接线

Q4 并行分析师隔离

Q5 多空辩论机制

Q6 风险三方辩论

Q7 记忆结算反思闭环

Q8 防未来函数

Q9 数据供应商路由

Q10 结构化输出与评级

Q11 怎么评价决策质量

Q12 中断续跑

Q13 能实盘吗

Q14 与回测引擎区别

一、疑问解答 (14 问)

Q1

TradingAgents 一句话是什么？

它把一次"该不该买这只股票"的决策，拆成一家虚拟交易公司里 12 个 LLM 员工的流水线协作：

4 个分析师分头调研 → 多空研究员互相辩论 → 交易员下提案 → 激进/保守/中性三人风险辩论 → 组合经理拍板。出自 arXiv 论文 2412.20138 (AAAI 2026)，作者 Tauric Research。

类比：你知道的[[多智能体协作]]是"几个 agent 分工干活"，TradingAgents 是把它做到金融决策上，并且**辩论 (debate) 是核心工序**——不是各写各的报告，而是真吵起来。

Q2

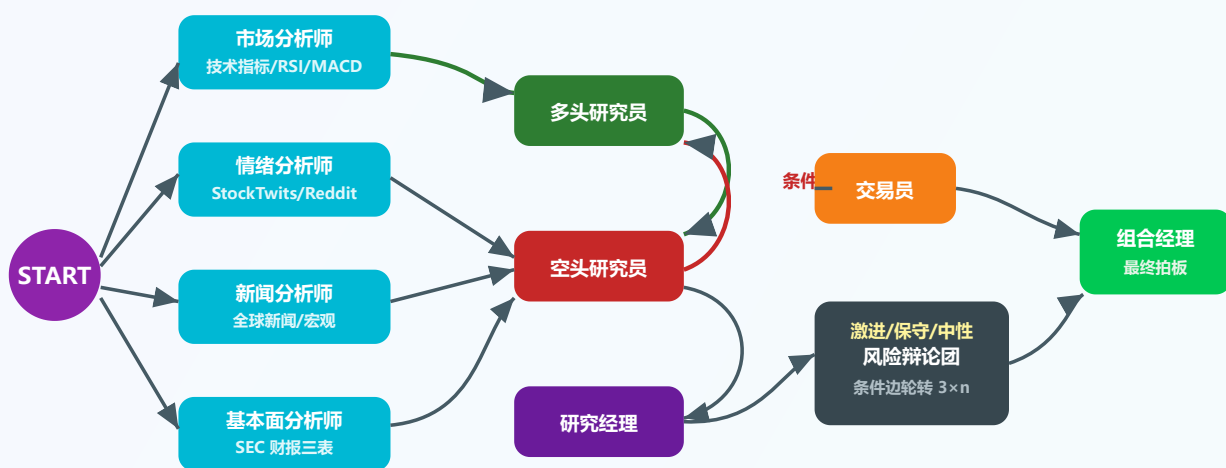
为什么要模仿交易公司的组织架构，而不是单 Agent 一次问答？

单 Agent 问一次"NVDA 能买吗"有三个毛病：①**视角单一**（一个 prompt 装不下基本面+情绪+新闻+技术面四套专业工具链）；②**没有对抗**（模型倾向于给一个自信的答案，没人唱反调）；③**无法审计**（错了不知道错在哪一环）。组织架构把"信息收集 / 观点形成 / 观点对抗 / 决策 / 风控"五件事强制分开，每一环都有独立报告留痕。论文实验显示多空辩论能显著减少单 Agent 的过度自信偏差。

Q3

12 个角色在 LangGraph 里是怎么接线的？

整个系统是一张 **StateGraph**（LangGraph 的状态图：节点=一个步骤，边=步骤间流转，全体共享一个状态字典）。实测拓扑（见实验 E2）：**16 个节点、29 条边**。接线逻辑：**START 同时分叉给 4 个分析师（并行）→ 全部完成后汇入 Bull Researcher → 条件边在多空之间来回 → Research Manager → Trader → 三个风险角色条件边轮转 → Portfolio Manager → END。**



🔗 已学过 **[[LangGraph与Agent编排]]**、**[[多智能体协作]]**：这张图就是它们的金融实战版。新看点是"分析师并行分叉 + 两段辩论环"的混合拓扑。

Q4

4 个分析师并行跑，怎么避免互相覆盖对方的结果？

这是 v0.5.2 最新改进。每个分析师不是直接挂进大图，而是编译成一个 **私有子图**：模型和工具调用在自己的私有消息历史里循环，最后只把报告文本写回共享状态的一个专属键（output_schema 只暴露一个字段）。所以并行线程互不可见对方的工具报文。

```
_analyst_graph(spec, agent, max_tool_rounds):  
    graph = StateGraph(AgentState, output_schema=TypedDict(..., {spec.report_key: str}))  
    # 工具轮超过 max_tool_rounds → 强制注入 WRAP_UP 消息要报告（防死循环打爆递归上限）
```

还有兜底：分析师赖着不停调工具，wrap_up 节点会数它调了几轮，超限就塞一条 HumanMessage 勒令交报告（issue #1420 的修复）。类比[[多Agent协作乱序竞态]]：解法同源——私有作用域 + 只回传约定字段。

Q5

多空研究员的"辩论"到底怎么实现？

没有魔法：就是一个 **条件边路由器 + 计数器**。共享状态里有 investment_debate_state，含 count（已发言次数）和 current_response（谁刚说完）。路由规则（实测，见实验 E4）：**Bull 说完→Bear 接；Bear 说完→Bull 接；count ≥ 2×max_debate_rounds → 交给 Research Manager 收官**。max_debate_rounds=1 时完整序列是 Bull→Bear→Bull→Research Manager（Bull 先手，所以多空各 1 次共 2 轮）。Research Manager 不是裁判打分，而是**写出一份综合双方论点的投资计划（investment_plan）**。

Q6

风险辩论为什么是三个人？和多头空头有何不同？

多空辩论辩的是**"该不该做"**（方向），风险辩论辩的是**"做了会怎样死"**（下行）。三个风险角色的立场是 **Aggressive 激进**（支持扩大敞口）、**Conservative 保守**（强调回撤/流动性）、**Neutral 中性**（对冲与执行细节）。路由同理：**Trader 固定先咨询 Aggressive，然后 Conservative→Neutral 轮转，count ≥ 3×max_risk_discuss_rounds → Portfolio Manager**。实测序列：Conservative→Neutral→PM（实验 E4）。三方出场保证风控不是一个人的独角戏——激进观点也必须被记录在案。

Q7

"决策 → 结算 → 反思"的记忆闭环怎么运转？

这是 TradingAgents 最有产品感的设计，一个 append-only 的 **markdown 记忆日志** 走三步：① **store_decision**：决策落盘，标签 [日期|代码|评级|pending]；② **结算**：持有窗口走完后，用真实行情算出 raw_return 和 alpha（相对区域基准的超额收益），原子写回标签变成 [+3.1% | +1.2% | 5d]；③ **Reflector**：LLM 把"决策+结局"浓缩成 2-4 句教训，下次分析同一只票时，这些历史教训会被注入 prompt——相当于这个框架有"经验值系统"。

```
[2026-01-09 | NVDA | Overweight | +3.1% | +1.2% | 5d] ← 实验E5真实输出
DECISION: BUY, Overweight...
REFLECTION: 教训：样本窗口太短，别把周线波动当日线信号。
```

重复写入同一(日期,代码)会被幂等挡掉(实测写 2 次只有 1 条)，结算用 temp-file + os.replace() 原子写，崩了不坏账本。类比[[JSONL事件日志与折叠模型]]：同样是只追加事件日志，只是这里存的是"决策事件"。

Q8

怎么防止"未来函数"（用未来数据做过去的决策）？

回测领域的经典毒瘤——你已知的[[事件驱动回测]]核心就是防它。TradingAgents v0.5.0 起全面落实 **point-in-time 完整性**：每一条带日期的数据路径都只返回"分析日当天或之前发布"的数据：财报按 SEC EDGAR 的**申报日**（不是报告期！）供数；FRED 宏观、社交媒体、新闻都有 look-ahead 过滤；回测网格里每个 (ticker, date) 单元只看得到该日期前公开的信息。连记忆日志都做 point-in-time：反思教训只来自"该日期之前已结算"的条目。

Q9

12 种数据需求，是怎么路由到 7 家免费数据源的？

vendor router：一张 `VENDOR_METHODS` 表把"方法名 → {供应商名: 函数}"映射起来，分析师只声明"我要 `get_fundamentals`"，路由层决定去 Yahoo 还是 SEC EDGAR 拿。实测 12 个方法全部被命中（实验 E3）：

数据	供应商
行情/OHLCV、技术指标、新闻	yfinance + stockstats（免费）+ Alpha Vantage
财报三表/内部人交易	SEC EDGAR（按申报日供数）+ yfinance
社交媒体情绪	StockTwits + Reddit
宏观指标	FRED（免费 key）
事件概率	Polymarket 预测市场

全部免费，跑通整个框架只需要一个 LLM 的 API key。数据缓存进本地（`data_cache_dir`），重跑不重复请求。

Q10

LLM 输出是自由文本，怎么保证"评级"解析得出来？

两层保险。① **结构化输出**：Research Manager / Trader / Portfolio Manager / 情绪分析师走 `with_structured_output`，直接吐 Pydantic 模型（枚举类型 `PortfolioRating` 从根上杜绝"评分格式五花八门"）；供应商不支持时自动降级自由文本。② **rating 完整性校验**：自由文本路径里用五档制

Overweight / Bullish / Neutral / Bearish / Underweight 正则硬解析，解析失败进 `review` 状态而不是悄悄变成别的信号。决策信号 `signal`、`state["final_rating"]`、记忆日志里的 `rating` 三处必须一致（实验 E3 断言验证过）。类比[[Agent输出协议契约]]。

Q11

它没有现金账本，怎么评价自己"决策得好不好"？

`backtest.py` 开头有一段非常诚实的 `scope` 声明："这评估的是决策质量，不是组合模拟器；把评级变成真实订单需要数量、成交价、现金账本，这里都没有；硬造一个就会把执行模型藏在评估工具后面"。所以它的回测是：同一个图跑 (`ticker`, `date`) 网格 → 每格一个决策 → 记忆日志就是结果表 → 用 `raw/alpha` 回报结算打分。各格独立，没有仓位滚动。这是少见的"知道自己边界在哪"的开源项目设计。

Q12

跑到一半 LLM 挂了怎么办？

LangGraph 的 `checkpoint (SQLite)` 持久化：每过一个节点存一次状态。LLM 调用失败抛异常后重跑，只补做没完成的调用，已完成的节点直接从存档恢复（repo 测试实测：fail_at=12 中断后续跑，新增调用数恰好等于全量调用数减去已完成数）。CLI 还支持 checkpoint resume 交互式续跑。类比[[Checkpoint存档与持久执行]]——同构思路：状态外置 + 断点续传。

Q13

我能拿它去实盘买股票吗？

不能，也不该。三重原因：①**框架定位**：官方 disclaimer 写明 research purposes only；②**工程缺口**：无仓位管理、无成交模拟、无手续费滑点（参见已学[[交易成本与鞭打效应]]——真实交易里这些足以吃掉全部 alpha）；③**模型局限**：决策质量随 backbone LLM、温度、数据质量剧烈波动，论文自己的回测也只是研究性质。它真正能给你的是一套可复用的“多智能体决策流水线”工程范式——把分析师/辩论/风控的角色拆分搬到任何需要“调研→对抗→决策”的领域都成立（竞品分析、投资尽调、技术选型）。

Q14

它和 NautilusTrader 这类回测引擎是什么关系？

完全互补，管的不同段。你已学的[[事件驱动回测]]（NautilusTrader）管“**执行层**”：纳秒级撮合、手续费、保证金、订单生命周期——回答“这个策略交易成本后还赚不赚钱”。TradingAgents 管“**决策层**”：回答“这个时段该看多还是看空”。TradingAgents 自己明说没有执行模型，所以真要严肃研究，产出信号后应该丢给 NautilusTrader 这类引擎去做含成本的真实回测——两者接起来才是完整闭环（决策→执行评估），这也是一个不错的进阶 DIY 方向。

二、这个项目给你什么（能力清单）

📁 你能拿走的东西：

- 一套多智能体辩论架构模板：并行子图 + 条件边辩论环 + 计数器终止，可直接抄到任何“需要对抗性决策”的 agent 项目里；
- 记忆-结算-反思闭环：给任何 LLM 应用加“经验值系统”的现成实现（markdown 日志 + 幂等 + 原子写）；
- point-in-time 数据纪律：任何回测/时序分析都要抄的防未来函数清单；
- 免费金融数据接入参考：yfinance / SEC EDGAR / FRED / StockTwits / Reddit / Polymarket 六路免费源的路由写法；
- 结构化输出工程：Pydantic 枚举 + 降级 + 一致性断言的决策解析范式；
- 离线可测试的 agent 图：ScriptedModel 假 LLM 全管线单测——agent 应用不烧钱也能 CI 的样板。

三、动手实验（全部真实运行，零 API key、零网络、零资金）

实验 E1 • 跑通官方离线端到端测试（环境验证）

操作：

```
cd F:\reminder\tradingagents
python -m venv .venv && .venv\Scripts\pip install .\repo pytest pytest-subtests
cd repo && ..\.venv\Scripts\python -m pytest tests/test_graph_end_to_end.py -q --
basetemp=..\exercise\pytest_tmp
```

目的：验证环境 + 亲眼看到“整个多智能体图不联网也能跑”。实现：repo 自带 ScriptedModel 假 LLM + 假数据供应商。

✅ 预期/实测结果：11 passed in 2.65s（含全管线决策、中断续跑、并行验证、报告流式 11 项）

⚠️ 坑：Windows 沙箱下 pytest 默认临时目录 %TEMP% 拒绝访问，必须 --basetemp 指到本地目录。

实验 E2 • 导出智能体图拓扑（16 节点 / 29 条边）

操作：

```
..venv\Scripts\python exercise\e2_graph_topology.py
```

目的：把 Q3 的架构图从“论文说法”变成“代码事实”。哑模型只用于接线，编译后用 LangGraph 的 `get_graph()` 反射拓扑，并 `draw_mermaid()` 导出。

✅ 实测结果：16 节点、29 边；START 分叉 4 分析师 → 汇入 Bull Researcher；多空/风险各一个条件边回环；mermaid 已存 exercise/graph_topology.mmd

实验 E3 • 主实验：离线跑通完整决策管线

操作：

```
.venv\Scripts\python exercise\e3_run_offline.py
```

目的：亲手复现一遍"4 分析师并行 → 辩论 → 交易员 → 风险辩论 → 组合经理拍板"全过程。实现：脚本化 LLM（先调通工具再交报告）+ 7 家供应商全换假数据，`propagate("NVDA","2026-01-09")`。

✓ 实测结果：`signal = final_rating = Overweight`；LLM 调用 15 次；分析师在 3 个并行线程（`ThreadPoolExecutor-1_0/1/2`）上跑过；12 个数据路由方法全部命中（`get_stock_data/get_indicators/get_news/get_global_news/get_macro_indicators/get_prediction_mar` 记忆日志落盘 pending 条目。断言通过。

⚠ 坑：VENDOR_METHODS 的 dict 直接迭代出的是供应商名（key），不是函数。

实验 E4 • 辩论路由器微实验（纯逻辑）

操作：

```
.venv\Scripts\python exercise\e4_e5_router_memory.py
```

目的：验证 Q5/Q6 的路由规则不是看源码想当然。

✓ 实测结果：`max_debate_rounds=1` → Bear→Bull→Research Manager（Bull 先手已开场）；风险辩论 → Conservative→Neutral→Portfolio Manager（3×1 计满）

⚠ 坑：模拟起点 count 要从 1 起算——Aggressive 的开场发言本身已计数，从 0 开始会多推出一轮 Aggressive。

实验 E5 • 记忆日志：存储 → 幂等 → 结算 → 反思

操作：同 E4 脚本后半段。目的：验证 Q7 的三步闭环。

✓ 实测结果：同一（2026-01-09, NVDA）写 2 次 → 只剩 1 条（幂等）；结算后标签 `[+3.1% | +1.2% | 5d]`、`pending→False`、REFLECTION 段原子写入。落盘 markdown 全文见实验输出。

⚠ 坑：结算 API 叫 `update_with_outcome`（不是 `settle_decision`，那是 `settlement.py` 里的编排函数）；alpha 落盘为格式化字符串 `" +1.2% "` 不是浮点数。

四、知识库互链

本项目沉淀 3 个新概念：[[私有子图输出隔离]]（并行分析师互不踩脚）、[[决策记忆结算反思闭环]]（agent 经验值系统）、[[对抗辩论式决策架构]]（多空/风险辩论的组织学）。

与已有概念互链：[[事件驱动回测]]（防未来函数同源、执行层 vs 决策层互补）、[[多智能体协作]] [[LangGraph与Agent编排]]（图接线）、[[多Agent协作乱序竞态]]（并行隔离）、[[Checkpoint存档与持久执行]]（断点续跑）、[[JSONL事件日志与折叠模型]]（append-only 日志）、[[交易成本与鞭打效应]]（为什么不能实盘）、[[AI交易终端]] [[个人量化工具箱范式]]（金融工具谱系）。

🌟 一句话总结：TradingAgents 的真正价值不是“帮你炒股”，而是给所有 agent 开发者示范了怎样把一个复杂决策拆成有对抗、有记忆、可审计的多智能体流水线。